



INDEPENDENT SECURITY REVIEW

August 20, 2026

LI.FI

FraxFacet (v1.0.0)

Independent Security Review

SECURITY RESEARCHER

Sujith Somraaj (somraajsujith@gmail.com)

REPORT PREPARED BY

Sujith Somraaj

Table of Contents

1	About Researcher	1
2	Disclaimer	1
3	Scope	1
4	Risk classification	1
4.1	Impact	1
4.2	Likelihood	2
4.3	Action required for severity levels	2
5	Executive Summary	2
6	Findings	3
6.1	Low Risk	3
6.1.1	initFrax is replayable despite being documented as one-shot	3
6.1.2	Tempo fee has no caller-enforced maximum	3
6.2	Informational	3
6.2.1	Ambiguous Tempo fee-token funding instructions	3
6.2.2	Generic immutable getters violate facet selector naming	3
6.2.3	Swap routes can consume pre-existing Diamond native balance	4

1 About Researcher

Sujith Somraaj is a distinguished security researcher and protocol engineer with over eight years of comprehensive experience in the Web3 ecosystem.

In addition to working as a Lead Security Researcher at Spearbit, Sujith is also the security researcher and advisor for leading bridge protocol LI.FI and also is a former founding engineer and current security advisor at Superform, a yield aggregator with over \$170M in TVL.

Sujith has experience working with protocols / funds including Coinbase, Solana Foundation, Layerzero, Edge Capital, Berachain, Optimism, Ondo, Sonic, Monad, Blast, ZkSync, Decent, Drips, SuperSushi Samurai, DistrictOne, Omni-X, Centrifuge, Superform-V2, Tea.xyz, Paintswap, Bitcorn, Sweep n' Flip, Byzantine Finance, Variational Finance, Satsbridge, Rova, Horizen, Earthfast, Angles and multiple other protocols.

Learn more about Sujith on sujithsomraaj.xyz or on cantina.xyz

2 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release, and does not give any warranties on finding all possible security issues of that given smart contract(s) or blockchain software. i.e., the evaluation result does not guarantee against a hack (or) the non existence of any further findings of security issues. As one audit-based assessment cannot be considered comprehensive, I always recommend proceeding with several audits and a public bug bounty program to ensure the security of smart contract(s). Lastly, the security audit is not an investment advice.

This review is done independently by the reviewer and is not entitled to any of the security agencies the researcher worked / may work with.

3 Scope

FraxFacet	src/Facets/FraxFacet.sol (v1.0.0)
IFraxHopV2	src/Interfaces/IFraxHopV2.sol (v1.0.0)

4 Risk classification

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

4.1 Impact

Rating	Definition
High	leads to a loss of a significant portion (>10%) of assets in the protocol, or significant harm to a majority of users.
Medium	global losses <10% or losses to only a subset of users, but still unacceptable.
Low	losses will be annoying but bearable — applies to things like griefing attacks that can be easily repaired or even gas inefficiencies.

4.2 Likelihood

Rating	Definition
High	almost certain to happen, easy to perform, or not easy but highly incentivized
Medium	only conditionally possible or incentivized, but still relatively likely
Low	requires stars to align, or little-to-no incentive

4.3 Action required for severity levels

Severity	Required action
Critical	Must fix as soon as possible (if already deployed)
High	Must fix (before deployment if not already deployed)
Medium	Should fix
Low	Could fix

5 Executive Summary

Over the course of 10 hours in total, LI.FI engaged with the researcher to audit the contracts described in section 3 of this document ("scope").

This is an differential review, and hence the scope is limited only to the changes introduced in the audit commit.

In this period of time a total of **5 issues** were found.

Project Summary

Project Name	LI.FI
Repository	lifinance/contracts
Commit	3ae7e5f
Audit Timeline	August 15, 2026 - August 20, 2026
Methods	Manual Review
Documentation	High
Test Coverage	Medium-High

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	2
Gas Optimizations	0
Informational	3
Total Issues	5

6 Findings

6.1 Low Risk

6.1.1 `initFrax` is replayable despite being documented as one-shot

CONTEXT: `FraxFacet.sol#L136`

DESCRIPTION: `initFrax` is documented as one-shot but does not revert when mappings are already initialized. The update script includes `initFrax` calldata in every non-empty `FraxFacet` upgrade, overwriting configured mappings. This can restore intentionally removed routes and emit a misleading repeated initialization event.

RECOMMENDATION: Add an `AlreadyInitialized` check and call `initFrax` only during the initial facet installation.

LI.FI: Acknowledged - the deploy script always seeds from `config/frax.json`, which is the source of truth.

RESEARCHER: Acknowledged.

6.1.2 Tempo fee has no caller-enforced maximum

CONTEXT: `FraxFacet.sol#L532`

DESCRIPTION: The `_sendViaTempo()` internal function fetches the TIP20 fee from the upgradeable HOP, then immediately pulls that amount from `msg.sender`. The only fee field in caller calldata, `nativeFee`, is explicitly ignored on Tempo.

Consequently, a fee spike, bad conversion quote, or malicious Hop upgrade can charge up to the caller's fee-token balance/allowance. This expands Hop's trust surface beyond transient Diamond balances: Hop can indirectly cause the Diamond to pull funds from the user's wallet.

RECOMMENDATION: Preserve the ABI by interpreting `nativeFee` as `maxFeeTokenAmount` on Tempo and revert when `quoteStatic(...) > nativeFee`. Add a test proving an excessive quote reverts before any fee-token transfer.

LI.FI: Fixed in `Ocf0cd0`

RESEARCHER: Verified fix.

6.2 Informational

6.2.1 Ambiguous Tempo fee-token funding instructions

CONTEXT: `FraxFacet.md?plain=1#L217`

DESCRIPTION: Documentation may be interpreted as requiring callers to pre-transfer the TIP20 fee token to the Diamond. The facet always pulls the quoted fee from the effective `msg.sender` using `transferFrom`. A pre-transfer can therefore remain stranded or cause the caller to fund twice when approval is also provided.

RECOMMENDATION: Clarify that the effective caller must hold and approve the fee token, and must not pre-transfer it to the Diamond.

LI.FI: Fixed in `e4aa5c1`

RESEARCHER: Verified fix.

6.2.2 Generic immutable getters violate facet selector naming

CONTEXT: `FraxFacet.sol#L48-L59`

DESCRIPTION: The public immutables `HOP`, `TIP_FEE_MANAGER`, `PATH_USD` generate generic Diamond selectors. Only `initFrax` is excluded by the `script/deploy/facets/UpdateFraxFacet.s.sol:18`.

No current collision was found, but these violate `[CONV:FACET-SELECTORS]` and create future facet-upgrade collision risk.

RECOMMENDATION: Prefer Frax-prefixed getters or exclude them from the Diamond cut.

LI.FI: Fixed in `9f7ec7e`.

RESEARCHER: Verified fix.

6.2.3 Swap routes can consume pre-existing Diamond native balance

CONTEXT: FraxFacet.sol#L264

DESCRIPTION: The swap entrypoint, `swapAndStartBridgeTokensViaFrax()` intentionally permits `nativeFee > msg.value` to support ERC20→native fee swaps and only reserves an absolute native amount; it does not prove that amount came from the current call. The fee is subsequently paid in the `_startBridge()` function.

If the Diamond already contains native currency, a caller can submit an unrelated token → Frax swap with `msg.value == 0` and have that existing balance fund their LayerZero fee.

The shared `SwapperV2` helper only refunds when the final balance exceeds the initial balance; it does not revert when the balance decreases.

RECOMMENDATION: Snapshot `address(this).balance - msg.value` before swapping and, before `send0FT`, require the balance to be at least `snapshot + nativeFee`. This still permits current-call `msg.value` and ERC20→native swap output.

LI.FI: Acknowledged, diamond does not hold funds per design.

RESEARCHER: Acknowledged.